# TSLA Stock Anomaly Analyzer


*Detect anomalous trading days and correlate price movements with market events*

**CS 5004 Final Project · Spring 2026**

Team SSS:

Yuqing Lei | Guoyue Liu | Wanjing Yang | Yingzi Zhou

# Design Rationale & Tools

## What does it do?

A Java desktop app that analyzes historical TSLA stock data, detects anomalous trading days, and correlates price swings with nearby market events — displayed through an interactive Swing GUI.

### Anomaly Detection

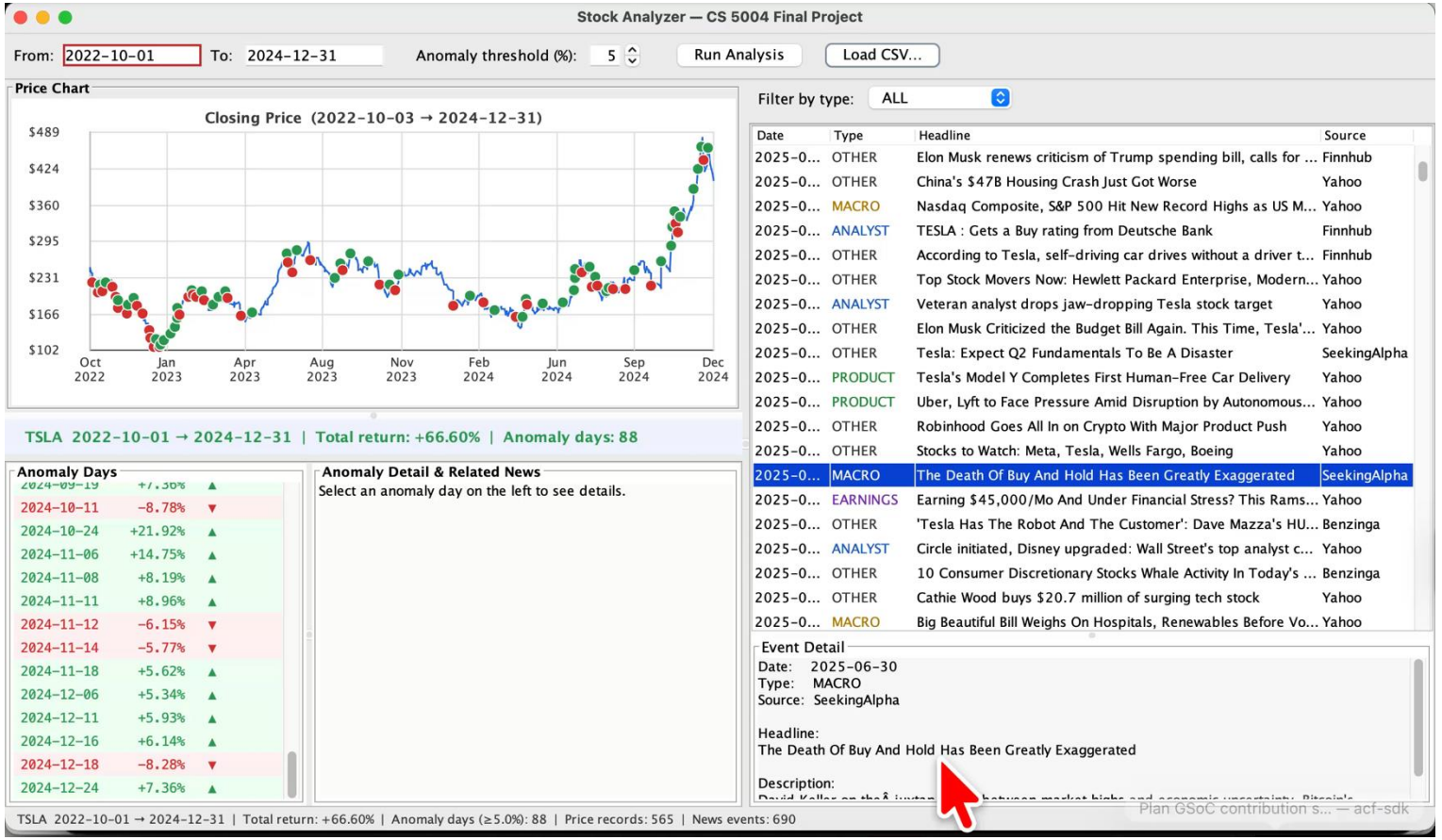
Flags days where absolute % change exceeds a user-defined threshold

### Event Correlation

Searches a ±3-day window around each anomaly for related news events

### Interactive GUI

Price chart, event table, and analysis summary — all in one Swing window




## Methods & Tools

**Java**
Core language with OOP/OOD focus

**Java Swing**
javax.swing for interactive GUI

# MVC Architecture

## Model

*Data & business rules*

**StockData**
Abstract base — date + validation helpers

- **PricePoint**
  One day's OHLCV data + % change
- **MarketEvent**
  News event with type, headline, source
  - **EarningsEvent**
    Extends MarketEvent with EPS fields
  - **StockDataModel**
    Aggregates prices + events per ticker
  - **AnalysisResult**
    Anomaly list, total return, summary

queries/ build →

## Controller

*Orchestration & parsing*

**StockController**
Anomaly detection, event correlation

**CsvLoader**
Parses price + news CSV files

pushes result →

← user action

## View

*Swing GUI components*

**MainFrame**
Top-level window, layout, controls

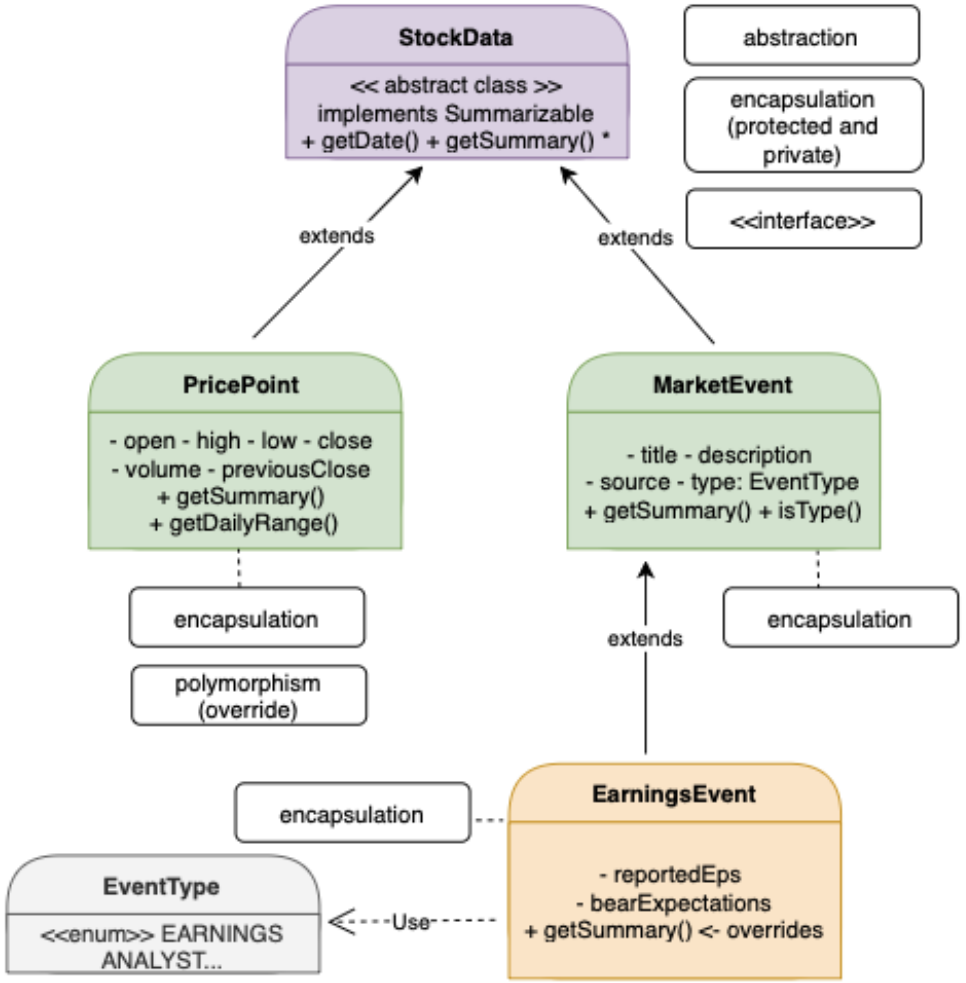
- **PriceChartPanel**
  Candlestick chart + anomaly markers
- **EventTablePanel**
  Scrollable, sortable news table
- **AnalysisPanel**
  Plain-text analysis summary

**Data Flow:**

CSV Files — CsvLoader — StockDataModel — StockController — AnalysisResult — View Panels

# MVC Architecture - Model

# Key OOD Concept – Interface and Abstraction




**Interface:** *Summarizable* defines the getSummary() contract, implemented by StockData so every subclass can be treated uniformly as a *Summarizable* type.

Summarizable.java

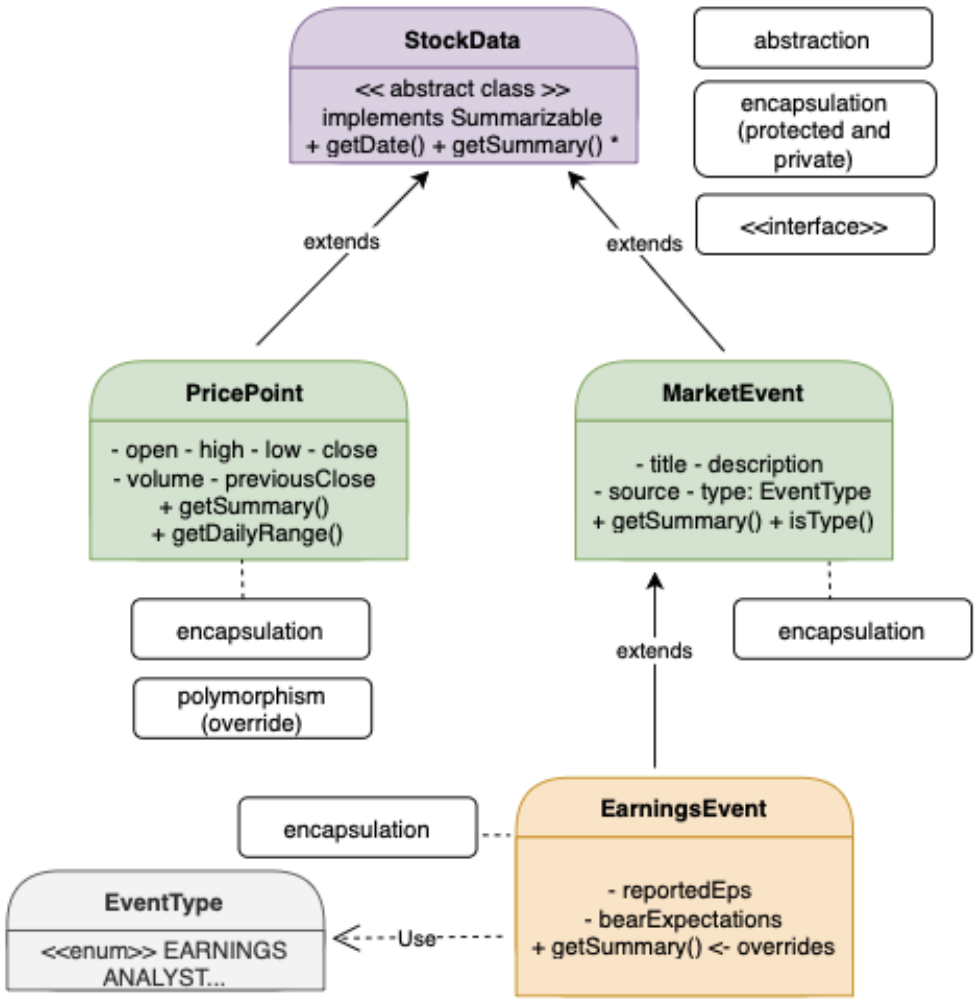
```java
public interface Summarizable {
  String getSummary();
}
```

**Abstraction:** *StockData* defines *getSummary()* as an abstract method, forcing all subclasses to provide their own implementation.

StockData.java

```java
public abstract class StockData {
  public abstract String getSummary();
```

# Key OOD Concept – Encapsulation and Inheritance




**Encapsulation:** Validation helpers like requireNonNull() are protected, accessible only to subclasses and hidden from outside. All fields in PricePoint and MarketEvent are private final, exposed only through getters.

StockData.java

```
protected static void requireNonNull(Object value, String fieldName) {...}
protected static void requireNonBlank(String value, String fieldName) {...}
protected static void requirePositive(double value, String fieldName) {...}
```

PricePoint.java

```
private final double open;
private final double high;
private final double low;
private final double close;
private final double previousClose;
private final long   volume;
```

MarketEvent.java

```
private final String    title;
private final String    description;
private final String    source;
private final EventType type;
```

**Inheritance:** *PricePoint* and *MarketEvent* both extend *StockData*, inheriting the date field and validation helpers without duplicating code.

PricePoint.java

```
public class PricePoint extends StockData {
```

MarketEvent.java

```
public class MarketEvent extends StockData {
```

PricePoint.java

```
public PricePoint(LocalDate date, double open, double high,
    double low, double close, double previousClose, long volume) {
  super(date);
```

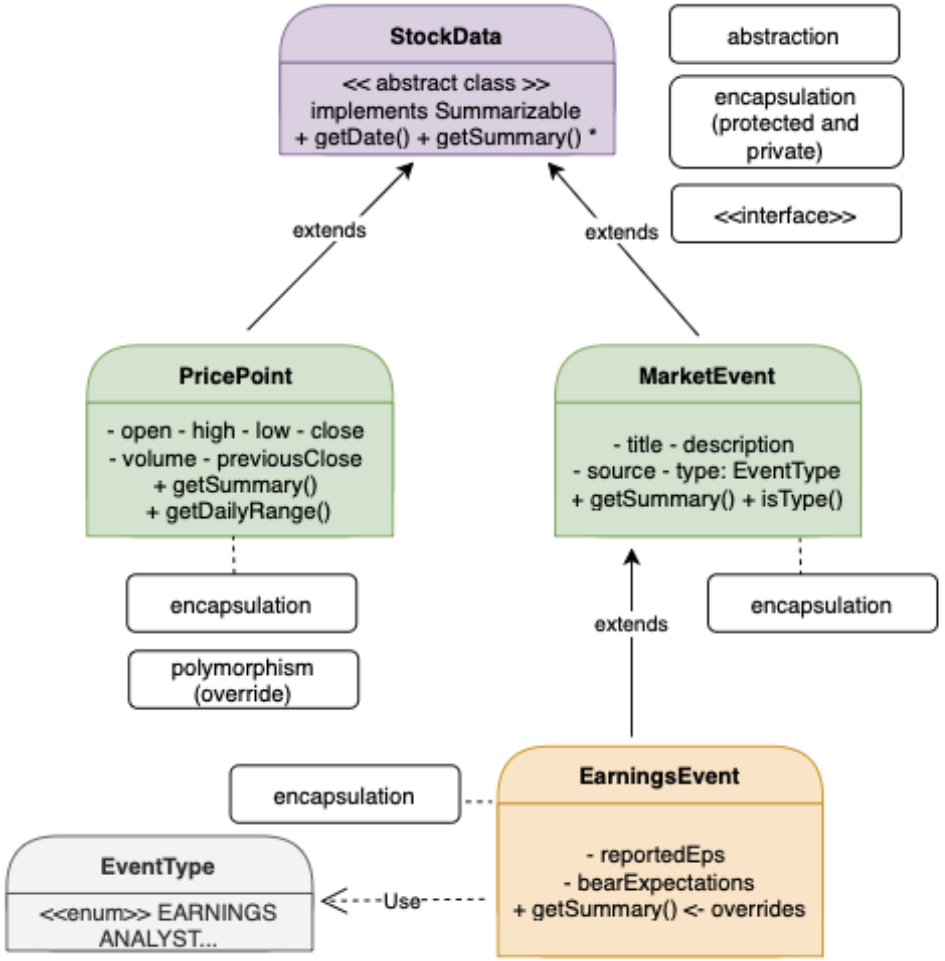
EarningsEvent.java

```
public class EarningsEvent extends MarketEvent {

  private final double  reportedEps;
  private final boolean beatExpectations;

  /** Constructs an EarningsEvent with earnings-specific fields. ...*/
  public EarningsEvent(LocalDate date, String title, String description,
      String source, double reportedEps, boolean beatExpectations) {
    super(date, title, description, source, EventType.EARNINGS);
    this.reportedEps      = reportedEps;
    this.beatExpectations = beatExpectations;
  }
```

# Key OOD Concept – Polymorphism and Dependency (Enum)




**Polymorphism:** Each subclass overrides *getSummary()* differently. *StockDataModel* iterates over *List<MarketEvent>* which can hold *EarningsEvent* objects, calling the correct *getSummary()* at runtime based on the actual object type.

PricePoint.java

```
@Override
public String getSummary() {
  return String.format("[Price] %s  close=%.2f  range=%.2f  vol=%d",
      getDate(), close, getDailyRange(), volume);
}
```

StockDataModel.java

```
public List<MarketEvent> getEventsByType(EventType type) {
  if (type == null)
    throw new IllegalArgumentException("EventType must not be null.");
  List<MarketEvent> result = new ArrayList<>();
  for (MarketEvent e : events) {
    if (e.isType(type)) result.add(e);
  }
  return result;
}
```

MarketEvent.java

```
@Override
public String getSummary() {
  return String.format("[%s] %s — %s (%s)", type, title, source, getDate());
}
```

EarningsEvent.java

```
@Override
public String getSummary() {
  String beat = beatExpectations ? "BEAT" : "MISSED";
  return String.format("[EARNINGS] %s — EPS: %.2f (%s) (%s)",
      getTitle(), reportedEps, beat, getDate());
}
```

**Dependency (uses):** *EarningsEvent* uses *EventType.EARNINGS* as a field type, depending on the enum without inheriting from it.

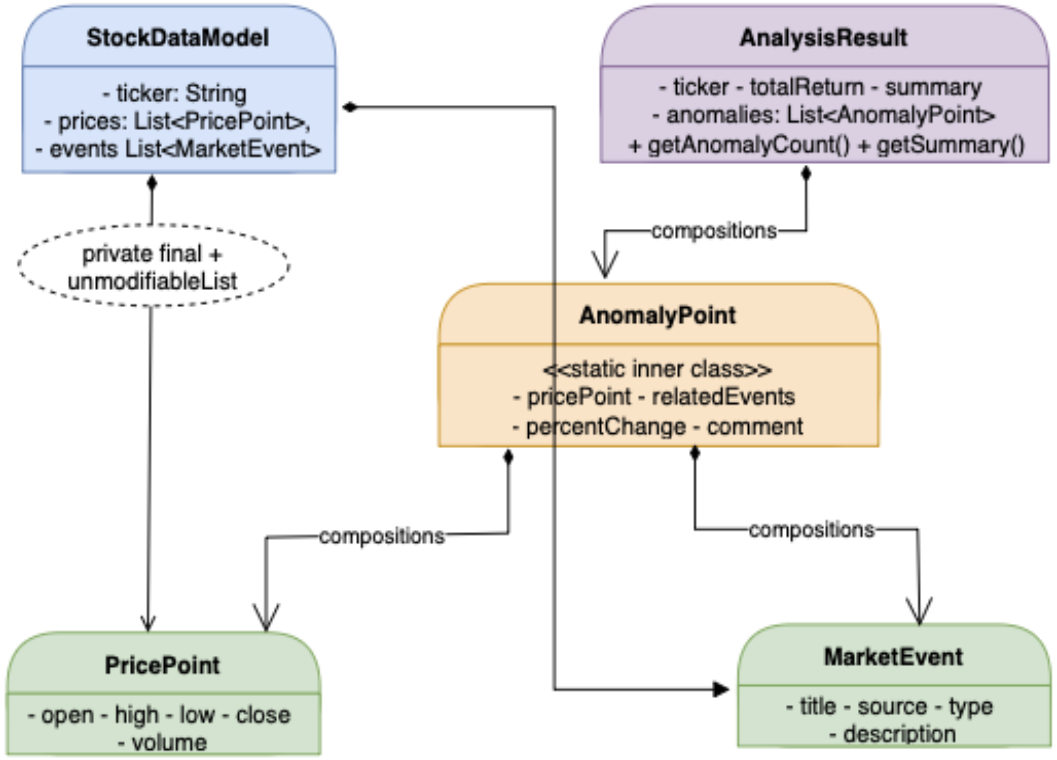
EventType.java

```
public enum EventType {
  EARNINGS, ANALYST, REGULATORY, MACRO, PRODUCT, OTHER
}
```

MarketEvent.java

```
public boolean isType(EventType eventType) { return this.type == eventType; }
```

# Key OOD Concept - Composition




**Composition:** *StockDataModel* owns *List<PricePoint>* and *List<MarketEvent>,* and *AnalysisResult* owns *List<AnomalyPoint>.* If the owner is destroyed, the owned objects are destroyed with it.

StockDataModel.java

```
private final String            ticker;
private final List<PricePoint>  prices;
private final List<MarketEvent> events;
```

AnalysisResult.java

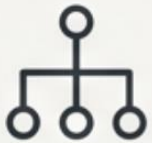
```
private final List<AnomalyPoint> anomalies;
```

AnalysisResult.java

```
private final PricePoint        pricePoint;
private final double            percentChange;
private final List<MarketEvent> relatedEvents;
private final String            comment;
```

# MVC Architecture - View

# Key OOD Concept in GUI

| | | |
|---|---|---|
|  | Encapsulation | **State protection.** `StockController` utilizes strict private fields. View classes are deliberately restricted to read-only access via explicit `getters`, preventing external state mutation. |
|  | Inheritance | **Base class extension.** All custom UI panels natively extend `JPanel`. Component renderers logically extend standard `Swing` base classes to ensure unified, predictable GUI behavior. |
|  | Polymorphism | **Dynamic rendering.** The system heavily relies on overridden methods—specifically `paintComponent`, `getListCellRendererComponent`, and `getTableCellRendererComponent`—to execute custom visual behaviors based on runtime data contexts. |
|  | Abstraction | **Contract-based data.** `EventTableModol` extends `AbstractTableModel`, effectively hiding complex underlying data structures from the UI and exposing them only through a standardized table contract. |
|  | Separation of Concerns | **Strict boundary enforcement.** File parsing is strictly isolated within `CsvLoader`, all business logic resides safely in `StockController`, and presentation code is locked exclusively within view classes. |

# MVC Architecture - Controller

# Our two data sources

## TSLA_price.csv

| | |
|---|---|
| Date | Trading date (2010–2026) |
| Open | Opening price |
| High | Daily high price |
| Low | Daily low price |
| Close | Closing price |
| Volume | Shares traded |
| % Change | Day-over-day percent change |

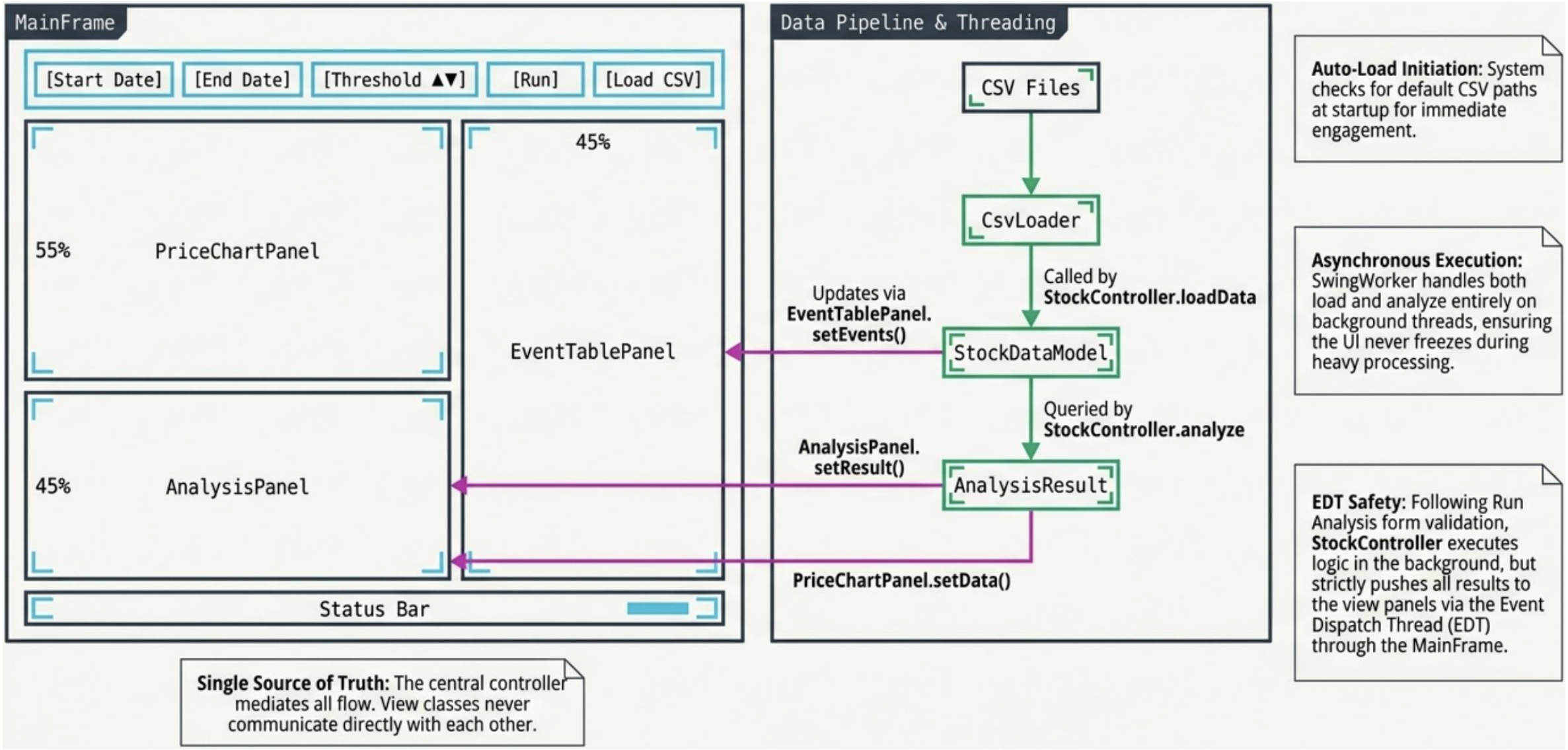
3,964 rows · Jun 2010 → Apr 2026 · Price: $1.05 – $489.88

## TSLA_news.csv

| | |
|---|---|
| Date | Publication date |
| Unix Time | Epoch timestamp |
| Headline | Article title |
| Summary | Brief description |
| Source | Publisher (Yahoo, Benzinga…) |
| URL | Original article link |
| Category | Event classification |

699 rows · Jun–Dec 2025 · Yahoo · Benzinga · SeekingAlpha

# Data pipeline in GUI



# Testing

**Test coverage summary**

| Test Class | Tests | What is covered |
|---|---|---|
| `PricePointTest` | 12 | Constructor validation, `getPreviousClose`, derived calculations, `getSummary`, `toString` |
| `MarketEventTest` | 11 | Constructor validation, `isType`, `getSummary`, `toString` |
| `EarningsEventTest` | 5 | Constructor, polymorphic `getSummary` override |
| `StockDataModelTest` | 17 | Constructor, defensive copy, `getPricesInRange`, `getEventsByType` |
| `AnalysisResultTest` | 21 | Constructor, `AnomalyPoint` inner class, unmodifiable lists |
| `StockControllerTest` | 10 | Pre-load state, guard clauses, empty-range analysis |
| **Total** | **76** | |

## Test Summary:

- Our test suite contains **76 JUnit 5 tests across 6 classes, covering the Model and Controller layers end-to-end**. Tests verify constructor validation, derived calculations, polymorphic behavior (calling getSummary() through supertype references), defensive copy enforcement (ensuring external mutation cannot corrupt internal state), and exception handling across MVC boundaries (using IllegalStateException for invalid program state vs. IllegalArgumentException for bad input).
- **The GUI layer was validated through manual exploratory testing**, as JUnit cannot drive Swing components.

**Polymorphism:** Calling getSummary() through a MarketEvent reference dispatches to EarningsEvent's override at runtime

```
@Test
void getSummary_viaMarketEventRef_usesOverride() {
  // Polymorphism: the overridden getSummary() should be called even via a MarketEvent reference
  MarketEvent ref = new EarningsEvent(DATE, "NVDA", "Desc", "Reuters", 1.62, true);
  assertTrue(ref.getSummary().contains("EPS"));
}
```

**Encapsulation/defensive copy:** Mutating the original list after construction has no effect

```
@Test
void constructor_mutatingOriginalList_doesNotAffectModel() {
  StockDataModel m = new StockDataModel("NVDA", prices, events);
  prices.add(new PricePoint(LocalDate.of(2025, 9, 5), 168.0, 169.0, 164.0, 167.0, 171.66,50_000L));
  assertEquals(3, m.getPriceCount()); // model still has original 3
}
```

**Exception handling across layers (MVC):** IllegalStateException (not IllegalArgumentException) used for wrong program state

```
// ── analyze: called before data is loaded ──────────────────────

@Test  no usages  Niuniu-Li-229
void analyze_dataNotLoaded_throwsIllegalState() {
  assertThrows(IllegalStateException.class, () ->
      controller.analyze(START, END, StockController.DEFAULT_THRESHOLD));
}
```

# Summary of Project

# Project highlights

### 🏗️ Clean MVC Architecture

Model, View, and Controller are fully separated. The controller drives all business logic; the view contains zero computation. Swapping the GUI for a different frontend would require no model changes.

### 🧬 OOP Principles Applied Throughout

Inheritance (StockData → PricePoint / MarketEvent → EarningsEvent), polymorphism (getSummary() override tested via supertype reference), encapsulation (private final fields + validation guards), and composition (StockDataModel, AnomalyPoint inner class).

### 🛡️ Robust & Defensive Data Layer

CsvLoader silently skips malformed rows so one bad line never aborts the load. All model objects are immutable after construction. Lists are wrapped in Collections.unmodifiableList() — callers cannot corrupt internal state.

### 📰 Automatic News Classification

CsvLoader classifies 699 news headlines into 6 EventTypes (EARNINGS, ANALYST, REGULATORY, MACRO, PRODUCT, OTHER) using keyword matching — no manual tagging needed.

### 📊 Fully Interactive Swing GUI

All three panels are visible simultaneously. The price chart renders custom-painted closing prices with anomaly dots, hover tooltips, and live coordinate mapping. Data I/O runs on SwingWorker background threads to keep the UI responsive.

### ✅ Thorough Unit Test Suite

5 JUnit 5 test classes cover all model classes. Tests include valid inputs, boundary conditions (zero volume, high == low, start == end), invalid-input exceptions, unmodifiable-list enforcement, and polymorphism verification.

# Limitation and Future Works

Data

## ⚠ Single stock, limited news window

Currently hardcoded to TSLA. News data only covers Jun–Dec 2025 (699 rows). Price history is available from 2010, but news context is missing for most of that period.

→ **Future:** Support multi-ticker comparison; integrate a live news API (e.g. Finnhub streaming) for real-time event feeds.

Data

## ⚠ Keyword classification is fragile

Headlines are classified by simple string matching (e.g. contains "earn" → EARNINGS). Ambiguous headlines or new phrasing can be misclassified as OTHER.

→ **Future:** Replace with an NLP classifier or fine-tuned sentiment model for more accurate and nuanced categorization.

Controller

## ⚠ Anomaly detection is threshold-only

A day is flagged as anomalous only if |% change| ≥ threshold. There is no statistical baseline (e.g. rolling volatility), so the same threshold applies equally to calm and turbulent periods.

→ **Future:** Use a dynamic threshold based on rolling standard deviation (Bollinger-band style) for context-aware anomaly detection.

Controller

## ⚠ No persistence or export

Analysis results exist only in memory during the session. Closing the app loses all results — there is no save, export to CSV, or PDF report feature.

→ **Future:** Add a 'Save Report' button that exports the AnalysisResult as a formatted PDF or CSV file.

View

## ⚠ Chart is read-only and non-zoomable

The PriceChartPanel renders a static line chart. Users cannot zoom into a sub-range, pan, or click a point to drill down — the only interaction is hover for a tooltip.

→ **Future:** Add zoom/pan via mouse wheel and click-to-select on the chart, passing the selected date back to the analysis panel.

Testing

## ⚠ No automated view tests

JUnit 5 cannot drive Swing components (no event dispatch thread, no rendered window), so all unit tests cover the Model and Controller layers only. The GUI was verified through manual exploratory testing — running the app and clicking through each feature by hand.

→ **Future:** Adopt AssertJ Swing or a headless UI framework to automate view-layer regression tests alongside the existing unit test suite.

# Findings and Lessons Learned (what is hard/easy)

## Wanjing

*Architecture and Testing*

### What was hard: Overall Project Structure

Deciding how to divide responsibilities across the MVC layers while keeping everyone's work aligned. When one part of the design changed, coordinating those changes across the team was difficult.

Our biggest concrete challenge: we agreed early on what each person would build, but we didn't establish a shared folder structure in GitHub. This caused version control conflicts when the frontend teammate tried to wire everything together. We resolved it by treating her integrated branch as the new main and archiving the original.

### Lesson Learned:

Agree on a clear project structure and a matching folder/branch convention in GitHub before anyone starts coding. Getting the scaffolding right upfront makes integration much smoother later.

## Yuqing

*Model*

### What was hard: OOD Concept throughout the model

Designing and applying OOD concepts consistently throughout the model layer. Deciding where to draw the line between classes. For example, whether EarningsEvent deserved its own subclass or could just be a field on MarketEvent required thinking carefully about inheritance vs. composition trade-offs. Getting the defensive copy pattern right (returning unmodifiable lists from every getter) also took iteration.

### Lesson Learned:

Start with clear interface contracts between classes before writing implementations. When the abstract base class and its method signatures are well-defined early, the subclasses fall into place much more naturally.

## Yingzi

*Data*

### What was hard: Data cleaning

Cleaning and aligning the raw data to match our model's expected format. The price CSV and news CSV had different date formats and inconsistent fields — some news rows were missing summaries, and the timestamps needed timezone normalization. Getting the data into a state where CsvLoader could parse it reliably took more effort than expected.

### Lesson Learned:

Invest time in understanding your data source before writing any parsing code. A quick exploratory pass to catch edge cases — missing fields, format inconsistencies, duplicate entries — saves a lot of debugging later.

## Guoyue

*GUI*

### What was hard: Learning by doing

Swing was not covered in class, so building the GUI was entirely learning by doing — reading documentation, experimenting with layout managers, and figuring out how to wire panels together. Getting the chart panel to render candlestick-style bars with anomaly highlighting was especially tricky since there's no built-in Swing component for that.

### Lesson Learned:

When working with an unfamiliar framework, build a minimal prototype first. Getting one panel to display dummy data end-to-end gave confidence and a working foundation before tackling the full layout with real data.

# Citations


- Oracle. (2024). Java Swing GUI Toolkit (javax.swing). Java Platform, Standard Edition Documentation.
- Yahoo Finance. (n.d.). Tesla, Inc. (TSLA) stock price, news, quote & history.
- JUnit 5 User Guide. junit.org/junit5/docs/current/user-guide/
- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns. Addison-Wesley.


# Thank You!

## Questions & Discussion


Yuqing · Guoyue · Wanjing · Yingzi


CS 5004 Final Project · Spring 2026